



Adaptive Pipeline Cache

Agenda

- Adaptive Window TinyLFU
- Pipeline Cache
- Adaptive Pipeline Cache
- Sematic Adaptive Pipeline Cache
 - Some issues with semantic caching



Window Tiny-LFU

- Combines both LRU and LFU policies in one policy.
- The LFU section uses TinyLFU to estimate the frequencies of items.
 - The implementation chosen is of SLRU.
- Widely used across the industry using the Caffeine library, among those:

- Cassandra
- Kafka
- Neo4j



Tiny LFU

- LFU needs knowledge regarding **all existing items in the universe**.
 - The frequencies do not age.
- TinyLFU solves this by using Count-Min Sketch.
 - Every X operations, we age all values in the sketch by dividing all values by two (right shift).

Count-Min Sketch

X

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	1
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	1
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

X

F_1	F_2	F_3	F_4
2	0	7	3
5	4	0	2
1	2	1	1
2	1	2	5
3	3	0	2
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- **Update only the minimal cells**
- Update all

On read, return the **minimal value**.

“Conservative Update”

Count-Min Sketch

X

F_1	F_2	F_3	F_4
2	0	7	3
5	4	0	2
1	2	1	1
2	1	2	5
3	3	0	2
0	3	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

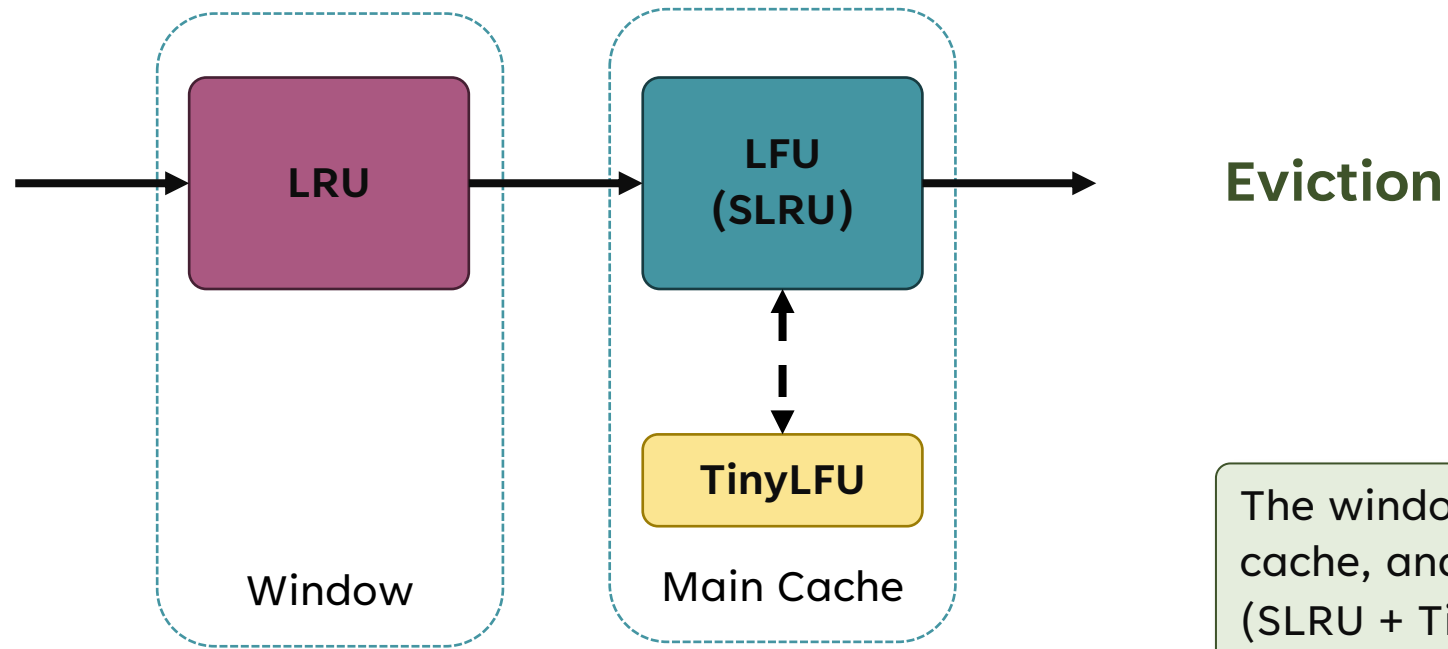
On write there are two options:

- Update only the minimal cells
- **Update all**

On read, return the **minimal value**.

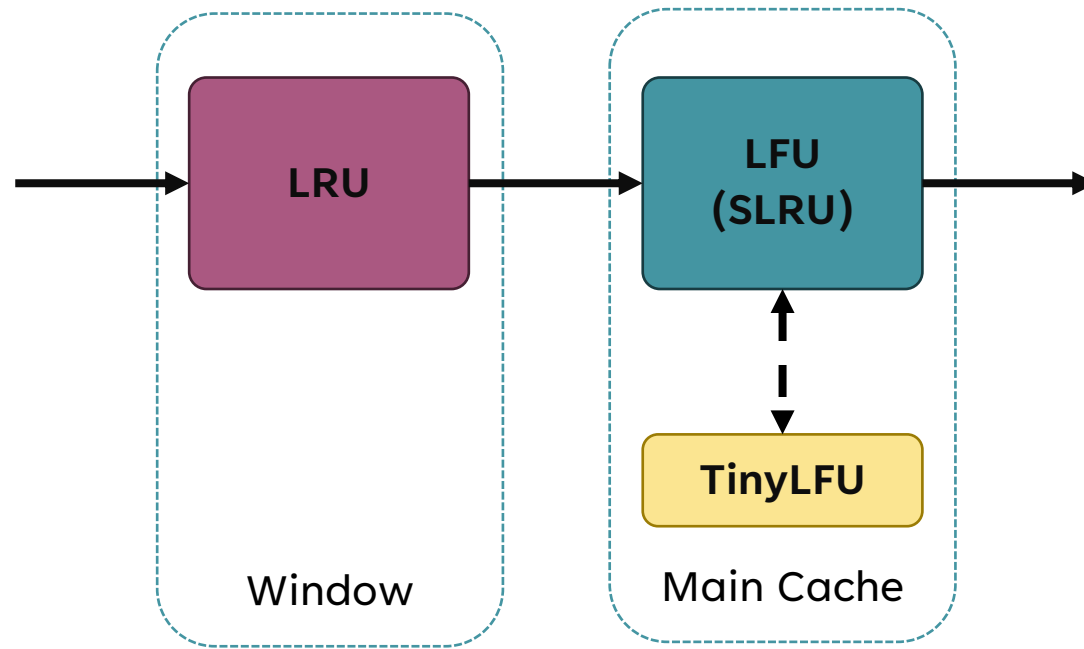
Update All

Window Tiny-LFU Architecture



The window (LRU) is 1% of the cache, and the main cache (SLRU + TinyLFU) is 99%

Adaptive Window Tiny-LFU



Eviction

Originally, the window (LRU) is 1% of the cache, and the main cache (SLRU + TinyLFU) is 99%

Every timeframe, 5% is transferred between the sections

Cost and Recency Aware Cache

- The **CRA** policy combines both the recency and the cost of items.
- The last access time of each item, t_i is stored in the cache.
- The score of an item is:

$$s(i) = c_i^{\frac{1}{(t-t_i+1)^k}}$$

for a regularization parameter k

- Implemented using multiple LRUs grouping items of the same cost range

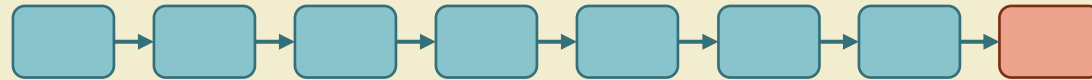
Cost and Recency Aware Cache

CRA

$[c_{min}, c_1)$



$[c_1, c_2)$



$[c_2, c_3)$



$[c_3, c_{max}]$



Cost-Aware Window TinyLFU

- The CA-W-TinyLFU is using CRA policy instead of the LRU blocks in both the window and the main cache, that is:
 - The window is just a CRA policy
 - The main cache is SLRU where both the probation and the protected are CRA policies.

Pipeline Cache



Caching with delayed hits

- Incorporating ***delayed hits*** into cache consideration is quite new
- The SoTA policy is **Latency-Aware Cache** (Yan and Li, ATC 2022)
 - Assumes that items are queried using a Poisson process
 - Performs eviction in time $\theta(M)$
 - Must save statistics for all the items in the universe I
 - Designed strictly to handle delayed hits

Our goal

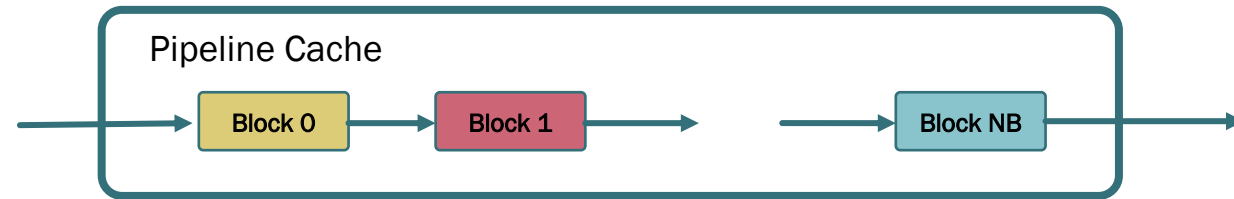
- Introducing a simple to implement yet time and space ***efficient*** policy for Delayed Hits.
- Incorporating this new policy easily into existing **tried-and-true** policies

Burst Cache

- A Priority Queue according to a burst estimation e_i
- Amortized time complexity of $\theta(\log M)$

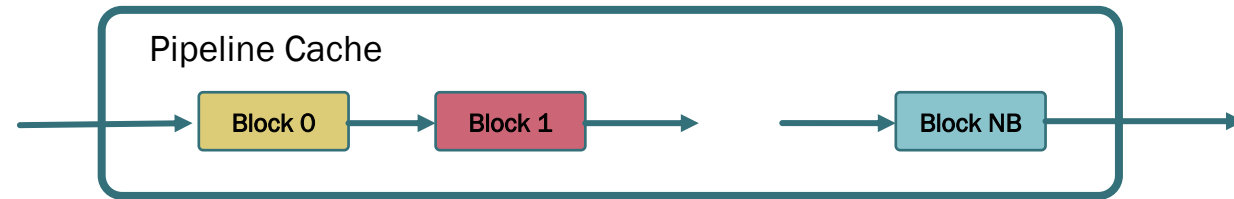
Incorporating with current solutions

- We wish to merge this heuristic with tried-and-true policies
- We introduce a new framework for combining different policies and heuristics
- The cache uses a pipeline architecture composed of stand-alone blocks
- The blocks are oblivious to the pipeline architecture and the other blocks

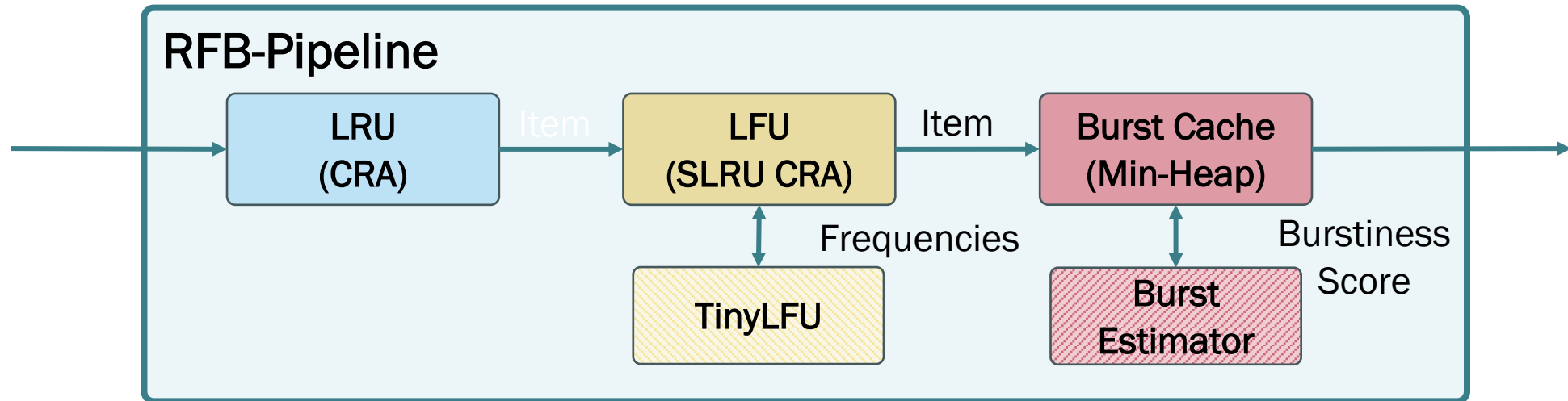


Incorporating with current solutions

- The block j has a quota q_j composed of quanta of size κ
- The total cache size is $M = \kappa \cdot Q$ for some natural number Q
- The pipeline order is predefined
- During the lifetime of an item within the cache, it is examined by all blocks



An Example: Our RFB-Pipeline



An example: Our RFB-Pipeline

LRU



$$q_R = 3$$

LFU



$$q_F = 10$$

BC



$$q_B = 3$$

$$Q = 16$$

$$\kappa = 64$$

$$M = Q \cdot \kappa = 1024$$

Adaptive Pipeline Cache

Adaptive approach

- Different traffic patterns call for different configuration
 - There is no “one size fits all”
 - Different configurations yield significantly different results
- Configuration is a “pain point”
 - Requires prior knowledge of the traffic
 - Static configurations yield poor results in highly dynamic traffic
- Specifically, burst cache is very situational

Simple hill climber

Current implementation in W-TinyLFU uses a simple Hill-Climber algorithm for adaptation

The climbing algorithm of W-TinyLFU is limited:

- Can handle only 2 blocks (LRU – LFU)
- Coupled algorithms
- Makes blind steps and evaluates the step benefit in hindsight
- Implicitly assumes the traffic does not change drastically

Adaptive pipeline cache

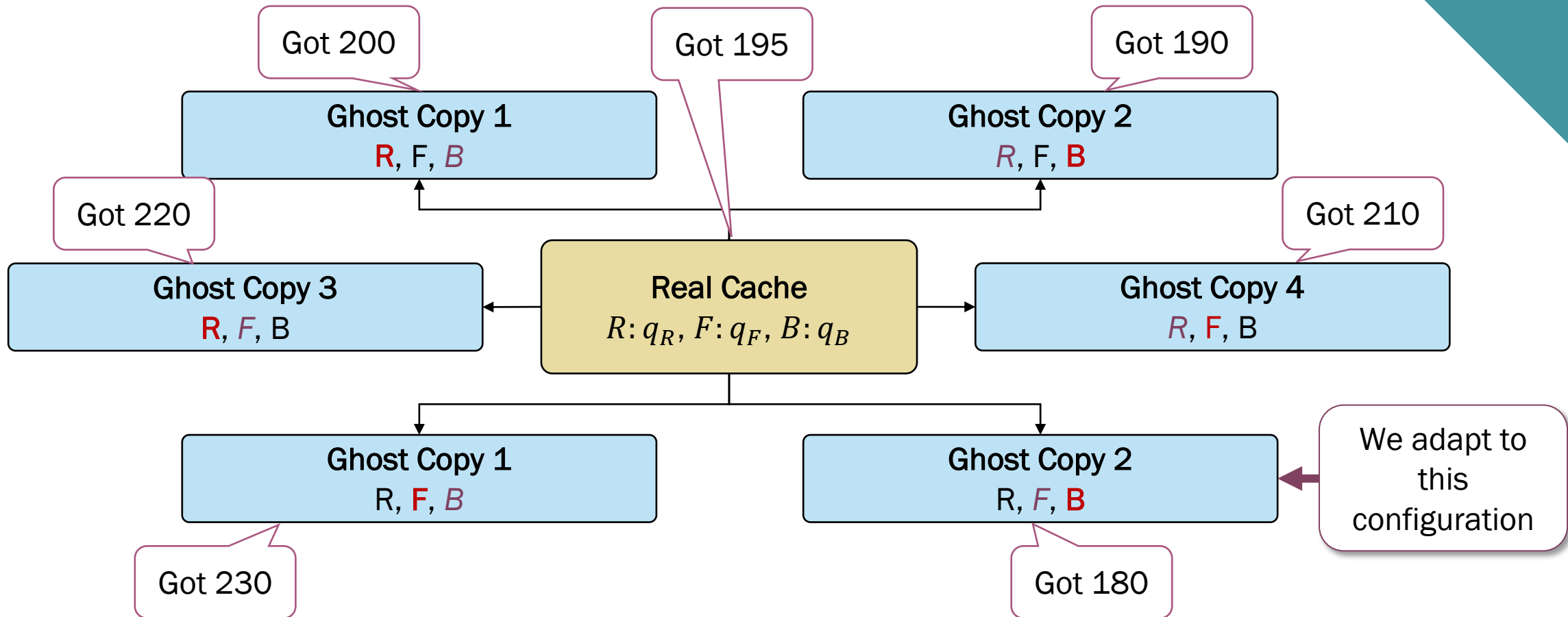
- Every predefined number of operations, the cache performs an adaptation
- In such adaptation, we move **one** quantum from one block's quota, to another block's quota
- In each adaptation stage, we have at most $(NB - 1) \cdot NB$ options for new configurations
- We can decide not to adapt if no benefit is gained

This is $2! \cdot \binom{NB}{2}$

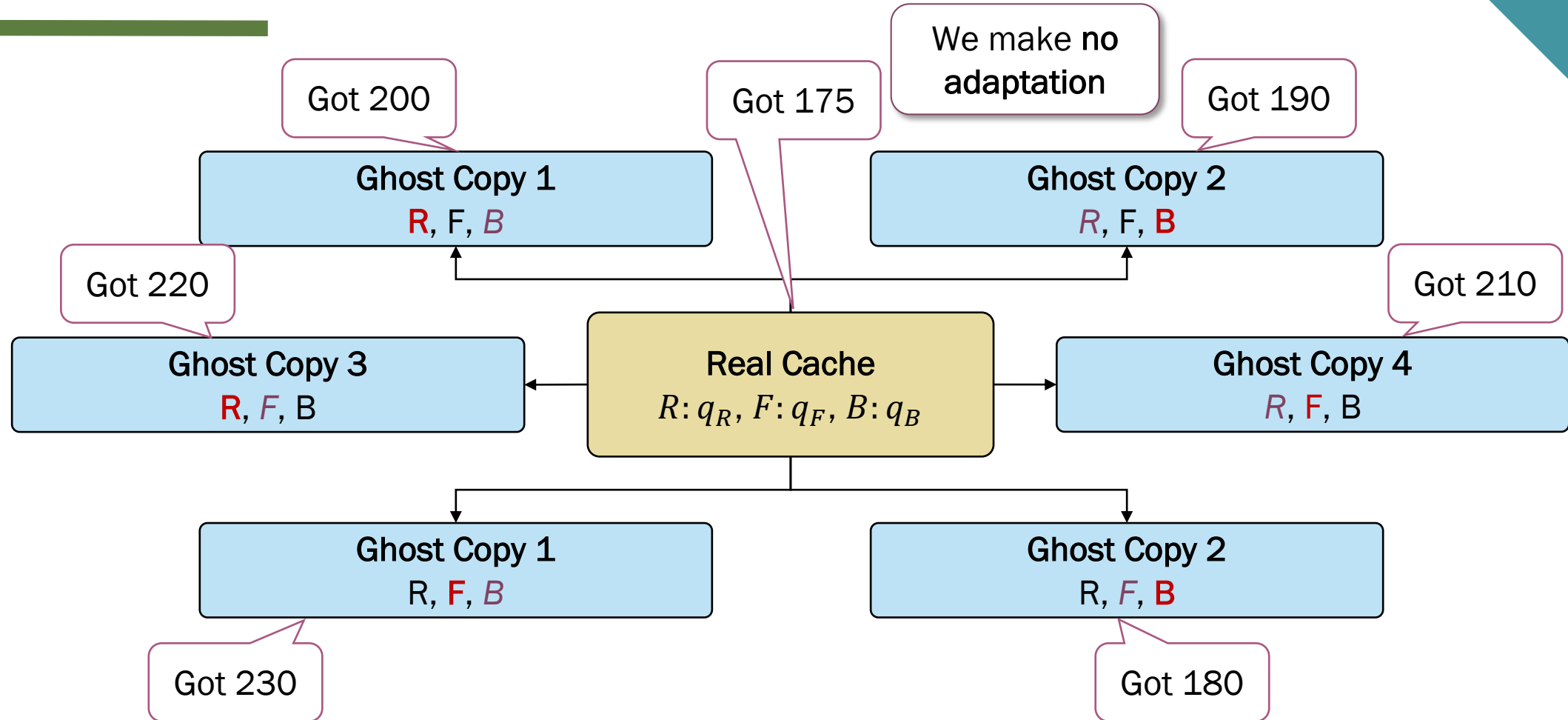
Full Ghost hill climber

- The new climber performs book-keeping for additional caches that contain only *ghost-entries* (without the actual data)
- At each stage, the algorithm simulated all possible cache configurations that are a single step from our current configuration
- At the end of each adaption window, we compare the performance of the actual cache, to the estimated performance of the ghost caches
- We change our configuration to match the best performing cache
- And iterate...

An example: our Rfb-pipeline



An example: our Rfb-pipeline



Full-ghost hill-climber

Pros:

- Can distinguish between performance deterioration due to poor choice and due to traffic change
- Responds to traffic changes rapidly
- When there is no traffic change, “converges” to a good configuration
- Can handle any number of blocks

Cons:

- Costly simulation of various states
- Meta-data overhead
- Can still fail due to local minima

Sampled Hill Climber

- Reduce the required ghost entries by handling only part of the traffic
 - If done correctly, can be captured by a cache with a size proportional to the part of the traffic considered
- At each stage, the algorithm simulated all possible cache configurations that are a single step from our current configuration
- Uses Spatial Hashing to decide whether a request should be performed in the simulation according to the item id

Semantic Adaptive Pipeline Cache



Caching and LLMs

- The major research in caching for LLMs focus on the following:
 - Caching intermediate results in Key-Value Caches on the LLM provider server
 - Usually on the GPU VRAM
 - Requires deep understanding of the model
 - Caching responses on the LLM service customer
 - Considers the LLM a black-box

Challenges in Semantic Caching

- Exact matches are rare, but different queries may be semantically the same
- The notion of *cache hit* and *cache miss* is no longer clear
- The most common approach is to:
 - Define a threshold $\tau \in [0,1]$
 - For request q , consider the nearest neighbor q'
 - There is a cache hit if $\text{sim}(q, q') > \tau$
- Now, we may have **false hits**
- There is no one similarity definition to rule them all

Challenges in Semantic Caching

- Many algorithms are not directly transferable to semantic environments
 - Window LFU is easy to implement
 - TinyLFU is not
- Hashing is quite complicated
 - Locality Sensitive Hashing (LSH)
- May introduce new localities in the traffic not present in non-semantic caching

Challenges in Semantic Caching

- Returning a response directly from the cache may be ill-advised
 - The result may be inaccurate even for a semantically identical queries
 - A privacy nightmare!
- The results may contain different media (text, code, images, video...)
- In general use, the topics are sparse (low caching potential)